Unity QuadSim File Overview

Project-authored source and configuration inventory

Scope. This document lists 102 project-authored code and text-based configuration files. Unity metadata, generated/cache/build output, documentation prose, binary libraries, textures, models, materials, and bundled dependency code are outside this inventory.

1. Package Definition

Defines how QuadSim is installed and compiled as a Unity package.

`package.json`
Unity Package Manager manifest defining the QuadSim package identity, Unity version, required Unity packages, distributable samples, and release metadata.

2. Runtime Assembly and Simulation Core

Contains the package assembly boundary and the simulation-wide clock, stepping, authority, configuration, catalog, and public API infrastructure.

`Runtime/Scripts/com.nino.quadsim.asmdef`
Compiles the runtime source into the Nino.QuadSim assembly and declares the UImGui assembly dependency used by the built-in interface.

`Runtime/Scripts/SimCore/ClockFactory.cs`
Creates and exposes the shared fixed-step and scaled clocks used throughout the simulation, including reset, pause, time-scale, and single-step operations.

`Runtime/Scripts/SimCore/ControlAuthorityManager.cs`
Tracks UI, internal-script, and external-client availability and grants one source command authority at a time, including the external connection approval flow.

`Runtime/Scripts/SimCore/DroneCatalog.cs`
Defines a ScriptableObject mapping YAML configuration names to QuadPawn prefabs, with a fallback prefab when no exact mapping exists.

`Runtime/Scripts/SimCore/FrequencyLimiter.cs`
Provides a simulation-clock-driven subrate scheduler so sensors or controllers can run below the base physics frequency without accumulating timing drift.

`Runtime/Scripts/SimCore/QuadSimEnvironmentBootstrap.cs`
Applies the selected scene configuration before drone spawning and RPC startup, forwarding ports, profile selection, spawn policy, frame, seed, geodetic origin, payload, mode, and wind settings to their owning systems.

`Runtime/Scripts/SimCore/QuadSimSceneConfig.cs`
Defines the researcher-facing ScriptableObject that stores scene-level run mode, drone profile, spawning, RPC endpoint, coordinate frame, sensor noise, geodetic origin, payload, and wind settings.

`Runtime/Scripts/SimCore/SimClock.cs`
Implements the authoritative nanosecond simulation clock, deterministic fixed-step clock, and pause/time-scale wrapper used instead of Unity wall-clock time.

`Runtime/Scripts/SimCore/SimCoreApi.cs`
Provides the public simulation-wide facade for pause, resume, stepping, time scale, reset, authority, connection state, and status retrieval.

`Runtime/Scripts/SimCore/SimulationManager.cs`
Owns manual Physics.Simulate execution and drives each registered ISimulatable through deterministic pre-physics and post-physics phases in free-run or manual mode.

`Runtime/Scripts/SimCore/StepContract.cs`
Validates requested target steps for strict lockstep calls and classifies missing, duplicate, mismatched, or skipped targets.

3. Shared Command Types

Defines the data shapes and axis conventions passed between command sources, APIs, controllers, and RPC handlers.

`Runtime/Scripts/SimCore/Common/CommonStructs.cs`
Defines shared source-status, controller, and goal-mode enumerations plus the Axis4, FlightCommand, Motor4, and Wrench6 value types used across the control path.

`Runtime/Scripts/SimCore/Common/FlightCommandLayout.cs`
Centralizes the meaning of the four command axes for position, velocity, angle, rate, wrench, and passthrough modes and provides validated command constructors.

4. External RPC Boundary

Implements the ZeroMQ and MessagePack boundary used by external clients while keeping Unity object access on the main thread.

`Runtime/Scripts/SimCore/Rpc/ExternalRpcAdapter.cs`
Runs the Unity-side RPC service, drains queued requests on the main thread, dispatches global and per-drone operations, manages connection state, and publishes telemetry, sensor, image, event, and learning data.

`Runtime/Scripts/SimCore/Rpc/RpcProtocol.cs`
Defines request, response, capability, and serialization helpers for the MessagePack wire protocol, including guarded parameter conversion and publish-message construction.

`Runtime/Scripts/SimCore/Rpc/RpcServerThread.cs`
Owns the background ZeroMQ request/reply and publish sockets and transfers requests, responses, publish frames, and log messages through thread-safe queues.

5. Drone Lifecycle, Composition, and API

Owns drone creation, registration, per-vehicle subsystem coordination, operating modes, and the stable per-drone API.

`Runtime/Scripts/DroneCore/Core/DroneManager.cs`
Maintains the active drone registry, selects and spawns configured prefabs, assigns stable identities, forwards scene settings, handles reload/reset/selection, and advances every drone during the simulation step.

`Runtime/Scripts/DroneCore/Core/ModeCoordinator.cs`
Tracks the active goal mode and controller kind and applies the required goal, controller, and plant reset policy when either changes.

`Runtime/Scripts/DroneCore/DroneRootBootstrap.cs`
Ensures a drone GameObject has exactly one required Rigidbody, pawn, controller, sensor, command, mode, and API component and applies baseline Rigidbody settings.

Runtime/Scripts/DroneCore/QuadPawn.cs
Coordinates one simulated vehicle by applying YAML configuration, initializing subsystems, running control and actuation before physics, updating sensors after physics, tracking collisions, and handling resets, cameras, and learning events.

Runtime/Scripts/DroneCore/Api/QuadSimApi.cs
Provides the stable per-drone interface for mode and controller selection, commands, sensors, telemetry, learning state, resets, frame changes, and configuration reloads.

6. Command Input and Mode Interfaces

Defines the boundary between command producers and the active drone control pipeline.

Runtime/Scripts/DroneCore/Interfaces/ICommandSource.cs
Defines the command-source contract used by controllers to read the current Axis4 goal and goal mode without depending on a specific UI, script, or network client.

Runtime/Scripts/DroneCore/Interfaces/FlightCommandProxy.cs
Accepts commands only from the authorized input source, records the active goal and writer, rejects unauthorized input, and supplies a safe fallback when needed.

7. Flight Controllers

Converts accepted goals and measured state into desired force, torque, and motor commands through the configurable control cascade.

Runtime/Scripts/DroneCore/Controllers/CascadedController.cs
Coordinates the position, velocity, angle, and rate stages; loads gains and limits; generates force and torque requests; invokes allocation; and exposes controller and allocation telemetry.

Runtime/Scripts/DroneCore/Controllers/AxisControllers/PositionController.cs
Implements one position-control axis, converting position error and its configured PID response into the next velocity-stage goal.

Runtime/Scripts/DroneCore/Controllers/AxisControllers/VelocityController.cs
Implements one velocity-control axis, using measured velocity and PID state to produce the acceleration or force-related goal consumed downstream.

Runtime/Scripts/DroneCore/Controllers/AxisControllers/AngleController.cs
Implements one attitude-angle axis, including nearest-equivalent angle handling, and produces the body-rate target for the rate stage.

Runtime/Scripts/DroneCore/Controllers/AxisControllers/AcroController.cs
Implements one body-rate control axis for acro/rate operation and returns the PID-generated torque demand.

8. Control Allocation and Physical Actuation

Maps controller demands to rotor commands and then to the physical wrench applied to the Unity Rigidbody.

Runtime/Scripts/DroneCore/Common/QuadPIDController.cs
Implements the reusable PID calculation, output limiting, integral decay, reset behavior, and downstream saturation tracking used by the flight-control stages.

Runtime/Scripts/DroneCore/Controls/ControlAllocator.cs
Builds the rotor effectiveness matrix and pseudo-inverse, allocates roll/pitch/yaw/collective demand to motors, applies desaturation priorities, and records rank, residual, and saturation diagnostics.

Runtime/Scripts/DroneCore/Controls/Quadactuationmodel.cs
Models asymmetric motor response lag, converts normalized motor commands into rotor thrust and reaction torque, and transforms the resulting body wrench for application in Unity world space.

9. Cameras and Image Capture

Provides player-facing camera control and drone-mounted image observations without changing the flight-control path.

Runtime/Scripts/DroneCore/Camera/CameraDirector.cs
Owns the active view camera and switches among follow, first-person, orbit, and free-camera behavior while binding to the selected drone and applying profile camera settings.

Runtime/Scripts/DroneCore/Camera/DroneCameraRig.cs
Stores the owning drone and the first-person, follow, and orbit mount transforms used by the camera director and image sensor.

Runtime/Scripts/DroneCore/Camera/DroneImageSensor.cs
Captures RGB, grayscale, or raycast depth observations at requested dimensions and encodings and packages them with agent, episode, step, time, frame, and sequence metadata.

10. Drone Debug Visualization

Supplies runtime and editor-only views of coordinate frames and rotor geometry for configuration verification.

Runtime/Scripts/DroneCore/Debug/DroneDebugDraw.cs
Draws the Unity body axes, selected output-frame axes, and optional motor axes around the live vehicle for coordinate-frame debugging.

Runtime/Scripts/DroneCore/Debug/RotorGeometryGizmo.cs
Draws YAML rotor positions, spin directions, center of mass, centroid residual, and differences from prefab motor mounts in the Unity Scene view.

11. Robot State and Sensor Scheduling

Owns the shared measured-state snapshot and coordinates deterministic post-physics sensor updates.

Runtime/Scripts/RobotCore/RobotCore.cs
Provides the per-drone RobotCore component and exposes the SensorManager discovered on the vehicle.

Runtime/Scripts/RobotCore/SensorData.cs
Defines the combined timestamped sensor snapshot containing IMU, GPS, magnetometer, barometer, validity, frame, episode, and step information returned to controllers and clients.

Runtime/Scripts/RobotCore/SensorManager.cs
Initializes the sensor suite from vehicle and scene settings, manages output frames and georeferencing, derives deterministic per-sensor seeds, schedules sampling, and publishes the latest SensorData after physics.

12. Sensor Models

Implements the individual simulated measurements consumed through SensorManager.

`Runtime/Scripts/RobotCore/Sensors/IMUSensor.cs`
Samples Rigidbody acceleration, body velocity, angular velocity, and orientation; converts them to the selected frame; and optionally applies reproducible measurement noise.

`Runtime/Scripts/RobotCore/Sensors/GPSSensor.cs`
Converts Unity position and velocity through the georeferencing system into local and geodetic GPS outputs with optional horizontal and vertical noise.

`Runtime/Scripts/RobotCore/Sensors/MagSensor.cs`
Combines vehicle attitude, geographic magnetic-field values, and the selected body frame to produce a simulated magnetometer reading and debug heading values.

`Runtime/Scripts/RobotCore/Sensors/BaroSensor.cs`
Produces pressure and altitude-related barometer output from the georeferenced vehicle altitude with optional repeatable noise.

13. Georeferencing, Magnetic Field, and Noise Utilities

Provides deterministic noise and geographic support shared by the sensor models.

`Runtime/Scripts/RobotCore/Core Utils/GaussianRng.cs`
Generates repeatable zero-mean Gaussian samples from a supplied seed for sensor-noise injection and episode reseeding.

`Runtime/Scripts/RobotCore/Core Utils/GeoReferencingSystem.cs`
Stores a WGS84 origin and converts between local east-north-up coordinates, latitude/longitude, and ellipsoidal height.

`Runtime/Scripts/RobotCore/Core Utils/GeoMagneticTables.cs`
Contains the maintained geographic lookup tables and interpolation functions used to obtain magnetic declination, inclination, strength, and the NED field vector.

`Runtime/Scripts/RobotCore/Core Utils/GeoMagDeclination.cs`
Defines an empty placeholder type; the current magnetic calculations are implemented in GeoMagneticTables rather than this file.

14. Coordinate Frames and Rotor Mathematics

Centralizes the frame conversions and rotor equations used by sensors, control allocation, actuation, APIs, and learning state.

`Runtime/Scripts/MathUtil/Frames.cs`
Defines supported simulation frames and the forward and inverse transformations for linear vectors, angular vectors, positions, accelerations, Euler attitude, and quaternions.

`Runtime/Scripts/MathUtil/RotorMath.cs`
Provides static thrust, yaw-torque, normalized-command, torque-convention, and first-order motor-response calculations shared by configuration, tests, and actuation.

15. Drone YAML Configuration

Loads vehicle profiles and maps their YAML fields into the strongly typed configuration consumed by the pawn, controller, sensors, and plant model.

`Runtime/Scripts/Yaml/Drone/DroneConfigLoader.cs`
Finds available vehicle profiles, gives host-project StreamingAssets files precedence over packaged defaults, and loads the selected YAML into a DroneConfig.

`Runtime/Scripts/Yaml/Drone/DroneConfigStructs.cs`
Defines the typed vehicle model, physical properties, rotor geometry, camera, controller gains, flight limits, obstacle settings, and derived rotor-physics structures represented by a profile.

`Runtime/Scripts/Yaml/Drone/YamlParser.cs`
Parses and validates schema-versioned YAML text into DroneConfig structures, including vectors, gains, physical parameters, rotors, cameras, and limits.

16. Runtime User Interface

Implements the built-in ImGui controls and diagnostics by calling the same APIs exposed to internal scripts and external clients.

`Runtime/Scripts/RuntimeUI/ImGui/ImGuiRuntimeBootstrap.cs`
Connects UImGui to the active simulation and camera, drains deferred UI operations, streams UI-owned commands when authorized, and coordinates all runtime panels and connection prompts.

`Runtime/Scripts/RuntimeUI/ImGui/TaskbarUI.cs`
Draws the top-level taskbar for simulation state and provides entry points to drone selection and the control, state, and settings panels.

`Runtime/Scripts/RuntimeUI/ImGui/ControlPanelUI.cs`
Maintains UI command state and draws the panel for authority, mode, controller, command-axis, reset, and related flight controls.

`Runtime/Scripts/RuntimeUI/ImGui/JoystickVizUI.cs`
Visualizes the current four-axis command as joystick-style controls using the selected drone, API, and UI command state.

`Runtime/Scripts/RuntimeUI/ImGui/PidSettingsUI.cs`
Displays and edits active cascaded-controller PID gains and renders the associated tuning-history window.

`Runtime/Scripts/RuntimeUI/ImGui/SettingsUI.cs`
Draws runtime settings for drone configuration and visibility of the control and state panels.

`Runtime/Scripts/RuntimeUI/ImGui/StateHudUI.cs`
Displays the selected drone's measured state, operating mode, telemetry, and related live status values.

`Runtime/Scripts/RuntimeUI/ImGui/SimImGuiStyle.cs`
Defines the restrained ImGui color and spacing theme plus reusable rounded-label and slider/input widgets.

`Runtime/Scripts/RuntimeUI/ImGui/UiActions.cs`
Queues UI-triggered actions and drains them outside the immediate-mode rendering callback so Unity state changes occur safely.

`Runtime/Scripts/RuntimeUI/ImGui/UiCommandState.cs`
Stores the editable position, velocity, angle, rate, wrench, and passthrough UI values and builds the Axis4 command for the active mode.

17. Reinforcement-Learning State and Scenario Objects

Builds step-aligned agent observations and manages externally described goals and obstacles.

`Runtime/Scripts/RL/RLAgentState.cs`
Captures normalized per-agent pose, velocity, angular state, collision state, and identifiers and defines the events drained from each pawn for learning clients.

`Runtime/Scripts/RL/RLSceneObject.cs`
Represents a configurable learning-scene object and applies its identifier, shape, collision setting, and display color.

`Runtime/Scripts/RL/ScenarioObjectManager.cs`
Creates, pools, updates, snapshots, and removes externally specified goals or obstacles for learning episodes.

18. Experimental Runtime Components

Contains optional prototype environment, payload, following, and trajectory-visualization behaviors built on the core APIs.

`Runtime/Scripts/Experimental/DroneFollower.cs`
Moves its GameObject above the currently selected drone and points it downward, providing a simple overhead follower behavior.

`Runtime/Scripts/Experimental/PayloadAttachment.cs`
Connects a payload Rigidbody to a drone mount with a ConfigurableJoint, matches initial motion to avoid an impulse, and updates an optional rope renderer.

`Runtime/Scripts/Experimental/TrajectoryVisualizer.cs`
Registers RPC visualization hooks and draws a requested path, desired-position ghost, velocity vector, and thrust-direction vector after FLU-to-Unity conversion.

`Runtime/Scripts/Experimental/windModule.cs`
Implements a configurable fan volume that applies simulation-time turbulence and velocity-relative drag once per Rigidbody per physics step, with optional force logging and editor gizmos.

19. Packaged Vehicle Profiles

Provides the default YAML vehicle configurations loaded when a host project does not supply an override.

`Runtime/Resources/QuadSim/Configs/Drones/crazyflie2_quad.yaml`
Defines the Crazyflie 2.1 mass, inertia, rotor placement, propulsion calibration, rates, gains, camera values, and flight limits.

`Runtime/Resources/QuadSim/Configs/Drones/flightmare_quad.yaml`
Defines the Flightmare-style quadrotor physical properties, asymmetric rotor placement, propulsion model, controller gains, camera values, and limits.

`Runtime/Resources/QuadSim/Configs/Drones/sentinel_quad.yaml`
Defines the ModalAI Sentinel/RB5 S500 profile using composed mass and inertia, calibrated rotor geometry and propulsion, asymmetric motor time constants, gains, camera values, and limits.

`Runtime/Resources/QuadSim/Configs/Drones/standard_quad.yaml`
Defines the standard larger quad profile with its physical properties, four-rotor geometry, propulsion constants, controller gains, camera values, and limits.

20. Runtime Visual Source

Contains the authored shader source used by a QuadSim environment visualization.

`Runtime/Prefabs/Models/WindZoneOutline.shadergraph`
Defines the configurable outline shader used to visualize the wind-zone volume, including outline color and size properties.

21. Unity Editor Tools

Provides authoring-time validation and visualization utilities that are excluded from player builds.

`Editor/Nino.QuadSim.Editor.asmdef`
Compiles the editor-only tools into an assembly that references the QuadSim runtime assembly and is restricted to the Unity Editor.

`Editor/DroneYamlInspectorWindow.cs`
Adds a QuadSim editor window that parses a selected vehicle YAML with the runtime parser and displays physical values, derived rotor performance, layout, effectiveness matrix, rank, and condition estimate.

`Editor/FindMissingScripts.cs`
Adds a menu command that scans all GameObjects in open scenes and reports missing component scripts with their hierarchy paths.

`Editor/Inspectors/RotorGeometryGizmoEditor.cs`
Adds inspector actions that compare YAML rotor positions with prefab motor mounts and can align the non-rendering mount transforms by geometric quadrant with Undo support.

22. Editor Tests

Verifies deterministic timing, protocol and YAML behavior, rotor physics, allocation, control limits, and packaged vehicle profiles without entering Play mode.

`Tests/Editor/Nino.QuadSim.Tests.asmdef`
Defines the editor test assembly, references the runtime package, and enables Unity's test assemblies only for the Editor platform.

`Tests/Editor/SimClockTests.cs`
Tests integer-derived fixed-clock time, reset epochs, and strict-step target classification.

`Tests/Editor/ProtocolAndYamlTests.cs`
Tests advertised protocol contracts, schema retention and rejection, request-method validation, and malformed MessagePack handling.

`Tests/Editor/RotorAndAllocatorTests.cs`
Tests rotor equations, motor lag, body/world wrench conversion, allocator rank and round trips, desaturation, anti-windup, desired-force constraints, angle wrapping, and validation of all packaged profiles.

23. Play-Mode Tests

Exercises the live Unity stepping contract and validates secure scene-configuration defaults.

`Tests/PlayMode/Nino.QuadSim.PlayModeTests.asmdef`
Defines the Play Mode test assembly and references both the QuadSim runtime and Unity test framework.

Tests/PlayMode/SimulationContractPlayModeTests.cs
Uses an ISimulatable probe to verify manual pre-physics/post-physics order, immediate stepping, reset behavior, and the default loopback RPC bind address.

24. QuadSim Starter Sample

Provides an importable example script, editable vehicle-profile overrides, and the sample assembly boundary.

Samples~/QuadSim Starter/Scripts/Nino.QuadSim.Samples.asmdef
Compiles the starter sample code into its own assembly and references the installed QuadSim runtime assembly.

Samples~/QuadSim Starter/Scripts/Internalapiexample.cs
Demonstrates internal-client initialization, authority acquisition, mode changes, rate and velocity commands, sensor and telemetry reads, reset operations, and simulation time controls through SimCoreApi and QuadSimApi.

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/crazyflie2_quad.yaml
Provides an editable StreamingAssets copy of the packaged Crazyflie profile so the sample project can override the package default.

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/flightmare_quad.yaml
Provides an editable StreamingAssets copy of the packaged Flightmare profile so the sample project can override the package default.

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/sentinel_quad.yaml
Provides an editable StreamingAssets copy of the packaged Sentinel profile so the sample project can override the package default.

Samples~/QuadSim Starter/StreamingAssets/Configs/Drones/standard_quad.yaml
Provides an editable StreamingAssets copy of the packaged standard-quad profile so the sample project can override the package default.

25. PX4 SITL Sample Tools

Contains optional command-line launchers and a MAVLink validation client for the PX4 SITL workflow.

Samples~/PX4 SITL Tools/test_offboard_figure8_3d.py
Runs a PX4 OFFBOARD three-dimensional figure-eight with position and rate-limited yaw commands, records MAVLink state, computes tracking metrics, produces plots and CSV output, and can broadcast live visualization samples over UDP.

Samples~/PX4 SITL Tools/scripts/run_bridge.sh
Launches the expected PX4-to-QuadSim bridge with localhost, port 4560, 250 Hz, and zero speed override arguments.

Samples~/PX4 SITL Tools/scripts/run_confirm.sh
Launches the expected PX4 confirmation helper with a three-meter takeoff target.

Samples~/PX4 SITL Tools/scripts/run_figure8.sh
Launches the figure-eight validation client at the standard speed and forwards live visualization packets to UDP port 14660.

Samples~/PX4 SITL Tools/scripts/run_viewer.sh
Launches the live viewer with the standard UDP port and plotting bounds used by the sample workflow.

26. Headless Visualization Tool

Provides a standalone visualization utility for simulations that run without the Unity game view.

`Documentation~/headless-tools/live_viewer.py`
Listens for UDP JSON path and sample packets and renders the planned path, actual trajectory, setpoint trail, current markers, and optional yaw-tracking status in an interactive three-dimensional Matplotlib view.

27. Continuous-Integration Utilities

Runs package validation against a built Unity player and supplies the minimal host-project dependency configuration used by automated tests.

`.github/workflows/phase3-ci.yml`
Defines the automated package contract checks, supported Python SDK test matrix, Unity Edit Mode and Play Mode tests, cross-platform player builds, built-player RPC smoke test, and validation artifact upload.

`.github/workflows/security.yml`
Defines the automated secret-scanning and dependency-review security gate for the repository.

`ci~/built_player_smoke.py`
Connects to a built player through the Python SDK, verifies loopback protocol negotiation and initialized drones, runs two seeded strict-lockstep sequences, compares state within tolerance, and writes a machine-readable validation manifest.

`ci~/check_unity_log.py`
Scans supplied Unity logs for compiler failures, runtime exceptions, protocol failures, and rejected validation conditions and returns a failing process status when any are found.

`ci~/UnityTestProject/Packages/manifest.json`
Defines the minimal Unity test host dependencies, installs QuadSim from the local package path, and marks the package as testable.